

Informe Retrospectivo — SmartLogix

Asignatura: Desarrollo Fullstack III — Sección 003V

Profesor: Miguel Acevedo

Integrantes: Juan Cerda, Jeremías Orellana

Evaluación Final Transversal — DSY1106

A. Diseño de la Arquitectura de Microservicios

Evolución de la arquitectura

La arquitectura de SmartLogix no fue una decisión tomada de una sola vez, sino el resultado de un proceso iterativo donde las necesidades reales del proyecto fueron redefiniendo las decisiones técnicas iniciales.

En una primera instancia, el equipo evaluó una arquitectura orientada a eventos basada en Apache Kafka. Esta opción parecía óptima en términos de escalabilidad asíncrona y desacoplamiento entre servicios, ya que un sistema de ecommerce logístico como SmartLogix involucra múltiples dominios (inventario, pedidos, logística) que podrían beneficiarse de comunicación por eventos. Sin embargo, al profundizar en la implementación, se identificó que Kafka añadía una complejidad operacional significativa: requería infraestructura adicional, conocimiento especializado en manejo de brokers, y una curva de aprendizaje que no se justificaba dado el alcance del proyecto.

La reflexión central fue que SmartLogix está orientado a PYMEs, y una arquitectura event-driven con Kafka representaba un exceso de ingeniería para la escala real del sistema. Esta toma de consciencia llevó al equipo a optar por microservicios con comunicación REST síncrona, lo cual entrega los beneficios de la separación por dominios y la escalabilidad progresiva, sin la sobrecarga operacional de un sistema de mensajería distribuida.

Arquitectura resultante

La solución final se basa en una arquitectura híbrida compuesta por:

- **Backend For Frontend (BFF):** punto único de entrada para el frontend, responsable de la autenticación, el enrutamiento y la inyección del contexto de tenant.

- **Microservicios independientes:** actualmente inventory-service, con orders-service y logistics-service proyectados como extensiones futuras.
- **Comunicación REST síncrona:** elegida por su simplicidad, facilidad de depuración y bajo costo operativo, adecuada para un MVP orientado a PYMEs.
- **Base de datos por servicio:** cada microservicio gestiona su propia base de datos, garantizando bajo acoplamiento e independencia de despliegue.

Alineación ética, sostenible y responsable

La arquitectura fue diseñada considerando principios de responsabilidad técnica:

- **Seguridad:** la autenticación JWT está centralizada en el BFF, evitando que cada microservicio deba implementar su propia capa de seguridad.
- **Privacidad:** el modelo de multi-tenancy con aislamiento por schema garantiza que los datos de cada empresa cliente sean completamente inaccesibles para otras.
- **Escalabilidad responsable:** se optó por una arquitectura que puede crecer progresivamente según las necesidades reales del negocio, evitando sobreinversión en infraestructura innecesaria para una PYME.
- **Sostenibilidad técnica:** el uso de Docker asegura portabilidad y consistencia entre entornos, reduciendo problemas de configuración y facilitando el mantenimiento a largo plazo.

B. Decisiones en el Desarrollo de Componentes Backend y Frontend

Backend

El backend fue desarrollado en Spring Boot, tecnología que el equipo ya manejaba y que ofrece un ecosistema maduro para la construcción de microservicios en Java. La separación por dominios dentro de cada servicio (módulos auth, product, config) facilitó la mantenibilidad del código y permitió que cada área fuera desarrollada y modificada de forma independiente.

Una decisión importante fue centralizar la seguridad y el enrutamiento en el BFF.

Inicialmente, la comunicación directa entre frontend y microservicios parecía más simple, pero el BFF resultó ser una capa valiosa que simplificó considerablemente el desarrollo del frontend, ya que este solo necesitaba interactuar con un único punto de entrada. Además, esta decisión deja abierta la posibilidad de incorporar múltiples frontends (web, móvil) apuntando al mismo backend en el futuro.

El uso de DTOs en cada capa fue una práctica que el equipo aplicó de forma natural, sin necesariamente identificarla como un patrón formal en ese momento. Retrospectivamente, se reconoce que el uso de DTOs contribuyó a desacoplar la representación de los datos en cada capa, protegiendo la integridad del modelo de dominio.

Frontend

La elección de Svelte como framework frontend fue una decisión deliberada basada en experiencias previas del equipo. En semestres anteriores, el uso de React en proyectos similares resultó en código difícil de mantener, con una curva de integración con el backend que consumió tiempo valioso. Svelte fue evaluado frente a Vue, y se optó por Svelte principalmente por su proximidad a JavaScript puro, lo que reduce la cantidad de conceptos propios del framework que el equipo debe manejar.

Técnicamente, Svelte presenta ventajas concretas: compila el código en tiempo de build en lugar de usar un virtual DOM en runtime, lo que se traduce en aplicaciones más livianas y rápidas. La separación entre componentes contenedor (lógica) y componentes de presentación (UI) fue aplicada siguiendo el patrón Container/Presentational, lo que facilitó la reutilización de componentes a lo largo de la aplicación.

Una mejora que se identificó durante el desarrollo fue la necesidad de estandarizar más el manejo de errores en las llamadas a servicios del frontend, lo cual podría haberse abordado de forma más sistemática desde el inicio.

C. Aplicación de Patrones de Diseño

Una de las reflexiones más significativas del equipo durante este semestre fue reconocer que varios patrones de diseño habían sido aplicados de forma intuitiva,

sin identificarlos explícitamente como tales en el momento de implementación.

Repository Pattern

Implementado en Spring Boot para abstraer el acceso a datos. El equipo lo adoptó como una forma natural de separar la lógica de negocio de las consultas a la base de datos, sin necesariamente identificarlo como “Repository Pattern” en ese momento. Retrospectivamente, se reconoce que esta separación facilitó la escritura de pruebas unitarias, ya que los repositorios podían ser mockeados de forma sencilla con Mockito.

DTO Pattern

Los DTOs fueron aplicados consistentemente entre capas y servicios. Al igual que el Repository Pattern, su uso surgió como una práctica que el equipo consideraba correcta antes de conocer su nombre formal. La reflexión retrospectiva fue que muchas buenas prácticas de ingeniería de software tienen nombre y están sistematizadas, y reconocerlas explícitamente permite comunicarlas mejor en equipo y aplicarlas más conscientemente.

Factory Pattern

Utilizado para la creación de tipos de envío, permitiendo instanciar distintas variantes de objetos logísticos sin acoplar el código a implementaciones concretas.

Container/Presentational (Frontend)

La separación entre lógica y UI en Svelte siguió este patrón, contribuyendo a la reutilización de componentes y a la claridad del código frontend.

Reflexión general sobre patrones

La experiencia de este semestre dejó como aprendizaje que los patrones de diseño no son reglas externas impuestas, sino soluciones a problemas recurrentes que emergen naturalmente cuando se programa con criterio. Identificarlos formalmente permite comunicar decisiones técnicas con mayor precisión y facilita la incorporación de nuevos desarrolladores al proyecto.

D. Estrategia de Branching y Gestión de Versiones

Evolución del flujo de trabajo

La estrategia de branching del proyecto siguió una evolución clara que refleja el proceso de maduración del equipo durante el semestre.

En las etapas iniciales del proyecto, durante el levantamiento de la arquitectura base y la configuración de los primeros microservicios, el equipo no aplicó de forma rigurosa el flujo de GitFlow. Los commits eran de mayor tamaño, agrupando conjuntos de cambios por cada funcionalidad terminada, y el trabajo se concentraba principalmente en la rama `develop`. Esta aproximación era comprensible dado que el proyecto aún estaba siendo definido estructuralmente, pero implicó que el historial de cambios en esa etapa fuera menos granular.

A medida que el proyecto se estabilizó, el equipo comenzó a aplicar ramas `feature/*` para cada nueva funcionalidad añadida, integrando los cambios a `develop` mediante merges controlados. Esta práctica mejoró la organización del trabajo y redujo los conflictos de integración.

Estado final del flujo

La estrategia GitFlow simplificada quedó estructurada de la siguiente forma:

- `main` → rama de producción, solo recibe merges estables
- `develop` → rama de integración continua
- `feature/*` → desarrollo de funcionalidades específicas
- `hotfix/*` → correcciones críticas en producción

En la práctica, las ramas `hotfix` no fueron necesarias porque las correcciones de errores se integraron directamente en las ramas `feature` correspondientes durante el desarrollo activo.

Impacto en el equipo

La adopción progresiva de GitFlow permitió que ambos integrantes pudieran trabajar en paralelo sin bloquearse mutuamente. La separación entre `develop` y las ramas `feature` facilitó la integración ordenada de cambios y redujo el riesgo de introducir código inestable en la rama principal.

Como mejora futura, se identifica que aplicar GitFlow con mayor rigor desde el inicio del proyecto, incluyendo commits más atómicos y descriptivos, habría

mejorado la trazabilidad del historial y facilitado la revisión de cambios.

E. Integración de Componentes Backend, Frontend y Base de Datos

Flujo de integración

La integración del sistema siguió el flujo definido por la arquitectura:

Frontend (Svelte) → BFF (Spring Boot) → Inventory Service → Base de datos (PostgreSQL)

El BFF actúa como intermediario, resolviendo el tenant desde el path de la URL, validando el JWT del usuario, e inyectando el contexto de tenant en las llamadas a los microservicios mediante el header X-Tenant-Id.

Multi-tenancy: el desafío más significativo

El desafío de integración más relevante del proyecto fue la implementación del sistema de multi-tenancy. El equipo evaluó múltiples alternativas:

- **Subdominios (empresaA.smartlogix.com):** descartado por la complejidad de configuración de DNS y certificados SSL en entornos de desarrollo y producción.
- **Bases de datos separadas por tenant:** descartado porque PostgreSQL ofrece aislamiento por schema sin necesidad de instancias separadas, lo que sería un desperdicio de recursos.
- **Filtrado por tenant_id en tablas compartidas:** descartado por el riesgo de fuga de datos y la complejidad del filtrado manual en cada consulta.

La solución adoptada fue **path-based routing + schema isolation en PostgreSQL**, donde cada tenant se identifica por el path de la URL y sus datos residen en un schema independiente dentro de la misma instancia de base de datos. Este enfoque entrega aislamiento fuerte a nivel de persistencia, evita el filtrado manual por tenant y mantiene la simplicidad en la capa de routing.

El flujo concreto es:

Frontend (/empresaA/api/products)

→ BFF (extrae tenant "empresaA" del path)

→ Inventory Service (recibe X-Tenant-Id, selecciona schema empresaA)

→ PostgreSQL (ejecuta consulta en schema correspondiente)

Desafíos enfrentados

La resolución dinámica del schema en la capa de persistencia fue la parte más compleja de la integración. Requirió configuración específica en Spring Boot para cambiar el schema activo en tiempo de ejecución según el tenant recibido. Las migraciones de base de datos también debieron adaptarse para aplicarse por schema, incrementando la complejidad del proceso de despliegue.

En la capa de integración frontend-BFF, el principal desafío fue estandarizar el manejo del tenant en todas las llamadas desde el frontend. Al estar el tenant embebido en el path de la URL, cada servicio del frontend debía construir las rutas incluyendo el identificador de tenant correspondiente, lo que requirió establecer una convención clara en la capa de servicios de Svelte para evitar inconsistencias. Esta necesidad reforzó la decisión de separar la lógica de llamadas a la API en una capa de servicios dedicada, en lugar de realizar las llamadas directamente desde los componentes.

Otro desafío fue la propagación correcta de errores desde los microservicios internos hacia el frontend a través del BFF. Se debió asegurar que los códigos HTTP (401, 403, 404, 409) fueran reenviados fielmente por el BFF sin ser absorbidos o transformados, de modo que el frontend pudiera reaccionar adecuadamente a cada caso. Esto quedó cubierto y verificado en la suite de pruebas del BFF.

F. Pruebas Unitarias y Aseguramiento de la Calidad

Contexto de implementación

Las pruebas unitarias fueron incorporadas al proyecto utilizando JUnit y Mockito en el backend, y Vitest en el frontend. Si bien no fueron parte del proceso de desarrollo desde el inicio bajo una metodología TDD, su implementación permitió verificar los contratos principales de cada servicio y detectar comportamientos no esperados en la lógica de negocio.

Evolución y retroalimentación

En una primera iteración, se alcanzó una cobertura global del 60%, distribuida de forma relativamente homogénea entre los tres servicios del backend. Sin embargo, durante la evaluación el profesor identificó que esta distribución no

era proporcional a la criticidad de los componentes: el BFF, por concentrar la superficie pública del sistema, la validación JWT, la resolución de tenant y el reenvío de requests hacia los microservicios internos, debería tener una cobertura significativamente mayor. Si el BFF falla, el frontend puede llamar a rutas incorrectas, saltarse validaciones de tenant o perder headers críticos antes de llegar a los servicios internos.

Esta retroalimentación llevó al equipo a reforzar la suite de pruebas, priorizando el BFF como componente de mayor peso.

Resultados tras el refuerzo

La línea base antes del refuerzo y los resultados actuales son los siguientes:

Servicio	Tests base	Tests actuales
BackendForFrontend	6/6 (OK)	22/22 (OK)
auth-service	5/5 (OK)	9/9 (OK)
inventory-service	7/7 (OK)	12/12 (OK)

La cobertura del BFF medida con JaCoCo alcanzó un **96.32% de instruction coverage** (786 de 816 instrucciones cubiertas), superando ampliamente la meta práctica de 70% establecida tras la retroalimentación.

Casos de prueba cubiertos

BackendForFrontend: - ProductControllerSecurityTests: validación de JWT en endpoints protegidos, verificación de que el tenant del token coincida con el tenant del path, rechazo de tenant cruzado, reenvío correcto del header Authorization, validaciones 400, y propagación de respuestas 401/403/404/409 desde los servicios internos. - InventoryClientTests: verificación de rutas internas /products, /products/{id}, /products/{id}/stock, métodos HTTP correctos, y envío de headers X-Tenant-Id y Authorization. - AuthServiceTests y AuthControllerTests: login y registro desde rutas públicas por tenant, verificación de headers y exposición correcta de respuestas del auth interno hacia el frontend.

auth-service: - Login con seed admin, password inválida, registro de usuario, usuario duplicado, tenant inexistente, header de tenant faltante, request inválido, y verificación del claim tenant en el JWT emitido.

inventory-service: - Autenticación requerida, header tenant obligatorio, creación y listado de productos, gestión de stock, aislamiento por tenant, SKU duplicado, producto inexistente, actualización completa, eliminación, producto inválido y stock negativo.

Impacto en la calidad del código

La escritura de pruebas evidenció la importancia del Repository Pattern adoptado durante el desarrollo: al tener el acceso a datos abstraído en repositorios, fue posible mockear las dependencias de persistencia con Mockito y probar la lógica de negocio de forma completamente aislada. Esto confirmó retrospectivamente que las decisiones de diseño intuitivas tomadas durante el desarrollo facilitaron directamente la testabilidad del código.

Las pruebas del frontend con Vitest se encuentran en proceso de implementación, complementando la cobertura ya consolidada en el backend.

Estrategia de mejora

Para futuras implementaciones, el equipo identifica las siguientes mejoras:

- Incorporar pruebas unitarias desde etapas tempranas del desarrollo, en paralelo con la implementación de cada funcionalidad.
- Definir umbrales de cobertura diferenciados por componente desde el inicio, priorizando módulos de mayor criticidad como el BFF.
- Incorporar pruebas de integración para validar el flujo completo entre BFF y microservicios, complementando las pruebas unitarias por servicio.

Conclusión

El desarrollo de SmartLogix representó un proceso de aprendizaje continuo donde las decisiones técnicas fueron evolucionando en respuesta a los desafíos reales del proyecto. La transición desde una arquitectura event-driven hacia microservicios con REST, la implementación del multi-tenancy por schema, y la adopción progresiva de GitFlow son ejemplos concretos de cómo el equipo fue ajustando sus decisiones en base a criterios técnicos y de viabilidad.

Retrospectivamente, uno de los aprendizajes más valiosos fue reconocer que muchas prácticas aplicadas intuitivamente durante el desarrollo corresponden

a patrones formales de diseño y arquitectura. Este reconocimiento no invalida las decisiones tomadas, sino que las dota de un marco conceptual que facilita su comunicación, justificación y mejora futura.

SmartLogix queda como una base sólida y extensible, sobre la cual los microservicios de pedidos y logística pueden incorporarse progresivamente sin requerir cambios estructurales en la arquitectura existente.